TABLE OF CONTENTS

1. Introduction 1
2. Background 2
3. Learning Objectives 3
4. Activities and Tasks 4
4.1 Task 1 – Analysis of Average Ratings and Total Reviews for Top App Categories 4
4.1.1 Objective 4
4.1.2 Data Handling 4
4.1.3 Filtering According to the Task 5
4.1.4 HTML Graph Code 6
4.1.5 Dashboard Code 7
4.1.6 Results and Interpretation 7
4.2 Task 2 – Global Distribution of Installs Using a Choropleth Map 8
4.2.1 Objective 8
4.2.2 Data Handling 8
4.2.3 Filtering According to the Task 9
4.2.4 HTML Graph Code 10
4.2.5 Dashboard Code 10
4.2.6 Results and Interpretation 11
4.3 Task 3 – Comparison of Free and Paid Applications Based on Installs and Revenue 11
4.3.1 Objective 11
4.3.2 Data Handling 12
4.3.3 Filtering According to the Task 13
4.3.4 HTML Graph Code 14
4.3.5 Dashboard Code 15
4.3.6 Results and Interpretation 15
4.4 Task 4 – Time-Series Analysis of Monthly Installs and Month-over-Month Growth 16
4.4.1 Objective 16
4.4.2 Data Handling 16
4.4.3 Filtering According to the Task 17
4.4.4 HTML Graph Code 18
4.4.5 Dashboard Code 19
4.4.6 Results and Interpretation 20

4.5 Task 5 – Bubble Chart Analysis of App Size, Rating, Installs and Sentiment Subjectivity 20
4.5.1 Objective 20
4.5.2 Data Handling 21
4.5.3 Filtering According to the Task 22
4.5.4 HTML Graph Code 23
4.5.5 Dashboard Code 24
4.5.6 Results and Interpretation 25
4.6 Task 6 – Cumulative Installs by Category Using a Stacked Area Chart 26
4.6.1 Objective 26
4.6.2 Data Handling 26
4.6.3 Filtering According to the Task 27
4.6.4 HTML Graph Code 29
4.6.5 Dashboard Code 30
4.6.6 Results and Interpretation 30
5. Overall Learning From the Six Tasks 33
6. Skills and Competencies 34
7. Feedback and Evidence 35
8. Challenges and Solutions 36
9. Outcomes and Impact 37
10. Conclusion 38

1. Introduction

The internship provided practical exposure to data analysis, data cleaning, visualization, and dashboard development using Python. The primary focus of the internship was to work with Google Play Store application data and transform raw datasets into meaningful visualizations that could be used to understand application performance, user engagement, ratings, installs, reviews, revenue, and other important characteristics.

Throughout the internship, six different data-analysis and visualization tasks were completed. Each task involved a different analytical problem and required specific filtering conditions, data transformations, visualization techniques, and dashboard requirements. The work was carried out mainly using Python libraries such as Pandas, NumPy, Matplotlib, Plotly, datetime, and pytz.

The internship helped bridge the gap between theoretical knowledge of Python and data analysis and their practical application. Instead of simply generating graphs, the tasks required understanding the dataset, cleaning inconsistent values, applying multiple conditions, aggregating data, selecting suitable visualization methods, and presenting the final results through HTML-based dashboards.

Another important aspect of the internship was the implementation of time-based visualization restrictions. Certain graphs were required to be displayed only during specified time periods in Indian Standard Time (IST). This introduced practical experience with conditional program execution and timezone-aware programming.

Overall, the internship provided an opportunity to develop an end-to-end data analysis workflow, beginning with raw data and ending with interpretable visualizations and dashboards.

2. Background

Data analysis is an important component of modern decision-making because large datasets often contain patterns and relationships that cannot be understood by simply examining raw values. Data visualization makes these patterns easier to interpret by presenting information through charts, maps, graphs, and dashboards.

The Google Play Store dataset contains information about mobile applications, including attributes such as app name, category, rating, number of reviews, size, number of installs, application type, price, content rating, Android version, and last updated date. A separate user reviews dataset also contains sentiment-related information associated with applications.

These datasets provide an appropriate environment for learning practical data analytics because they contain numerical, categorical, textual, and date-based variables. They also contain values that require preprocessing before analysis. For example, install counts contain symbols such as commas and plus signs, while application sizes can be represented in megabytes or kilobytes.

The internship project therefore focused on developing different analytical views of the Play Store data. The tasks progressed from category-level comparisons and geographical visualization to more complex analyses involving revenue, time-series data, sentiment subjectivity, bubble charts, month-over-month growth, cumulative installs, category translations, and conditional dashboard display.

The overall workflow followed during the internship can be represented as:

Raw Dataset → Data Cleaning → Data Transformation → Filtering → Aggregation → Analysis → Visualization → Dashboard Generation

This workflow reflects a commonly used structure in practical data analytics projects.

3. Learning Objectives

The main objective of the internship was to develop practical skills in Python-based data analysis and visualization. The internship was designed to strengthen the ability to convert raw datasets into useful analytical outputs while following specific business or analytical requirements.

The major learning objectives were:

1. To understand the structure and characteristics of real-world datasets.
2. To perform data cleaning and preprocessing using Pandas.
3. To handle missing, duplicate, invalid, and inconsistent data.
4. To convert columns into appropriate numerical and datetime data types.
5. To create reusable functions for data transformation.
6. To apply simple and complex filtering conditions using Python.
7. To perform category-wise and time-wise data aggregation.
8. To use groupby(), agg(), pivot(), cumsum(), and percentage-change calculations.
9. To merge information from multiple datasets.
10. To create different types of visualizations using Matplotlib and Plotly.
11. To develop interactive and static HTML-based graphical outputs.
12. To create dashboards containing analytical summaries and visualization links.
13. To implement time-dependent visualization restrictions using IST.
14. To understand how different visualization techniques should be selected according to the analytical problem.
15. To improve debugging, logical reasoning, and problem-solving abilities while working with Python.

4. Activities and Tasks

During the internship, six data-analysis and visualization tasks were completed using the Google Play Store dataset and, where required, the User Reviews dataset. The tasks progressively introduced data cleaning, filtering, aggregation, geographical visualization, revenue analysis, time-series analysis, sentiment analysis, cumulative calculations, interactive dashboards, and time-dependent graph restrictions.

4.1 Task 1 – Analysis of Average Ratings and Total Reviews for Top App Categories

4.1.1 Objective

The objective of Task 1 was to create a grouped bar chart comparing the **average rating** and **total number of reviews** of the top ten Google Play Store application categories according to total installs.

- The task required the following conditions to be applied before visualization:
- Application size must be **10 MB or greater**.
- The application must have been **last updated during January**.
- The category must have an **average rating of at least 4.0**.
- After filtering, only the **top ten categories according to total installs** should be displayed.
- The graph should only be generated and accessible between **3:00 PM and 5:00 PM IST**.

The visualization therefore aimed to compare two different indicators of application performance: average user satisfaction represented by rating and the volume of user feedback represented by reviews.

4.1.2 Data Handling

The Google Play Store dataset was loaded into apps_df. The User Reviews dataset was also loaded as reviews_df, although it was not required for the final Task 1 analysis.

Rows containing missing values in important columns such as App, Category, Rating, Reviews, Size, Installs, and Last Updated were removed.

The Installs column contained commas and plus signs. These characters were removed before converting the column to integer format.

A custom function named parse_size_to_mb() was developed to standardize application sizes. Values expressed in megabytes were directly converted to floating-point numbers, while kilobyte values were divided by 1024.

```
def parse_size_to_mb(val):
    val = str(val).strip()

    if val.endswith('M'):
        return float(val[:-1])

    elif val.endswith('k') or val.endswith('K'):
        return float(val[:-1]) / 1024.0

    return None
```

The Rating and Reviews columns were converted into numeric values, while Last Updated was converted into a Pandas datetime object.

This preprocessing ensured that mathematical comparisons and aggregations could be performed correctly.

4.1.3 Filtering According to the Task

Applications were initially filtered according to size and update month:

```
filtered_df = apps_df[
    (apps_df['Size_MB'] >= 10.0) &
    (apps_df['Last Updated'].dt.month == 1)
]
```

The remaining applications were grouped by category and three values were calculated:

```
category_df = (
    filtered_df.groupby('Category').agg(
        Total_Installs=('Installs', 'sum'),
        Avg_Rating=('Rating', 'mean'),
        Total_Reviews=('Reviews', 'sum')
    ).reset_index()
)
```

The category-level average rating was then used to retain categories with an average rating of at least 4.0.

Finally, categories were sorted according to total installs and the first ten were selected:

```
top10_df = category_df[
    category_df['Avg_Rating'] >= 4.0
].sort_values(
    by='Total_Installs',
    ascending=False
```

).head(10)

The filtering process therefore ensured that the graph represented only qualifying January-updated applications of sufficient size and high-rated categories.

4.1.4 HTML Graph Code

A grouped bar chart was created using Matplotlib.

Average rating was represented on the primary Y-axis:

```
rects1 = ax1.bar(
    x - width / 2,
    top10_df['Avg_Rating'],
    width=width,
    color='royalblue',
    label='Avg Rating'
)
```

Total reviews were represented using a second Y-axis:

```
rects2 = ax2.bar(
    x + width / 2,
    top10_df['Total_Reviews'],
    width=width,
    color='darkorange',
    label='Total Reviews'
)
```

A secondary axis was necessary because ratings are restricted to approximately 1–5, whereas review counts can reach millions.

The graph was generated only when:

is_within_time = start_time <= current_time < end_time

where the permitted period was 3:00 PM–5:00 PM IST.

The Matplotlib graph was stored in memory using BytesIO(), converted into PNG format and encoded using Base64.

```
img_buffer = io.BytesIO()
plt.savefig(img_buffer, format='png', bbox_inches='tight')
img_buffer.seek(0)

img_base64 = base64.b64encode(
    img_buffer.getvalue()
).decode('utf-8')
```

The encoded image was inserted into an HTML page:

```
<img src="{img_base64}"
    alt="Matplotlib Graph">
```

The resulting file was saved as task_1_graph.html.

4.1.5 Dashboard Code

The dashboard was generated as task_1_dashboard.html.

A summary card described the filters:

```
<div class="card">
  <h2>Dashboard Summary</h2>
  <p>
    <b>Filter Rules Applied:</b>
    App Size ≥ 10 MB, Updated in January,
    and Category Average Rating ≥ 4.0.
  </p>
</div>
```

Another section displayed the top ten categories in tabular form, including:

- Category name
- Average rating
- Total reviews
- Total installs

The graph-status section depended on the current IST time. If the time requirement was satisfied, an **Open Graph** button was displayed. Otherwise, the dashboard informed the user that the visualization was restricted.

4.1.6 Results and Interpretation

The final visualization compares the average rating and total reviews of the ten categories with the highest total installs after all filtering conditions have been applied.

The rating bars are expected to remain relatively close to one another because categories with an average rating below 4.0 were already removed. In addition, Google Play ratings have a maximum value of 5, naturally limiting the amount of variation.

Review counts, however, can vary by millions. Therefore, considerably larger differences can occur in the review bars.

A category can have a very large number of reviews without necessarily having the highest average rating. This is because these variables measure different aspects of application

performance. **Average rating reflects user satisfaction, whereas total reviews primarily reflect the volume of users providing feedback.**

The results should therefore be interpreted only for applications satisfying the size and January-update conditions rather than as a representation of the complete Play Store dataset.

4.2 Task 2 – Global Distribution of Installs Using a Choropleth Map

4.2.1 Objective

The objective of Task 2 was to create an interactive **choropleth map** representing the global distribution of installs for the top five qualifying application categories.

The task required:

- Categories beginning with **A, C, G, or S** to be excluded.
- Categories to be ranked according to total installs.
- Only the **top five categories** to be retained.
- Categories having more than **1,000,000 installs** to be highlighted.
- Global installs to be represented geographically.
- The visualization to be displayed only between **6:00 PM and 8:00 PM IST**.

4.2.2 Data Handling

Rows missing Category or Installs were removed.

The Installs values were converted from strings such as:

1,000,000+

into numerical values such as:

1000000.

This was achieved by removing commas and plus signs and converting the remaining value to integer format.

The data was then grouped by Category, and installs belonging to applications in each category were summed.

Because the Play Store dataset used in the project does not contain actual country-level install information, the notebook defined a separate set of 30 countries and associated weights.

These country weights were normalized:

```
countries_df['Weight'] = (
    countries_df['Weight'] /
    countries_df['Weight'].sum()
```

```
)
```

Each selected category was then cross-joined with the 30 countries:

```
merged_df = top5_categories.merge(
    countries_df,
    how='cross'
)
```

Estimated country-level installs were calculated as:

```
merged_df['Country_Installs'] = (
    merged_df['Installs'] *
    merged_df['Weight']
).astype(int)
```

Therefore, the geographical values in this task represent a **weighted distribution created by the notebook**, rather than observed country-level install counts from the original Play Store dataset.

4.2.3 Filtering According to the Task

The category-level totals were first calculated:

```
category_df = (
    apps_df.groupby('Category')['Installs']
    .sum()
    .reset_index()
)
```

Categories beginning with A, C, G, or S were excluded:

```
filtered_df = category_df[
    ~category_df['Category'].str.startswith(
        ('A', 'C', 'G', 'S')
    )
]
```

The five categories with the greatest install totals were selected:

```
top5_categories = filtered_df.nlargest(
    5,
    'Installs'
)
```

A Boolean field was also created to identify categories exceeding one million installs:

```
top5_categories['Highlight'] = (
    top5_categories['Installs'] > 1000000
)
```

4.2.4 HTML Graph Code

Plotly's go.Choropleth() was used to create the interactive map.

```
go.Choropleth(
    locations=cat_data['ISO'],
    z=cat_data['Country_Installs'],
    text=cat_data['Country'],
    colorscale=color_scales[i],
    colorbar_title='Installs'
)
```

A separate choropleth trace was created for each of the five categories.

A dropdown menu was then implemented so the user could switch between categories.

Categories exceeding one million installs were labelled with:

(>1M)

The Plotly graph was directly exported to:

```
fig.write_html('task_2_graph.html')
```

Unlike Task 1, Base64 conversion was unnecessary because Plotly can generate interactive HTML directly.

4.2.5 Dashboard Code

The Task 2 dashboard included:

- Explanation of the 30-country weighted distribution.
- Excluded category letters.
- The >1M highlight rule.
- Combined installs of the top five categories.
- A table containing the top five categories and total installs.
- Graph availability status.

The visualization was generated only when the current IST time was between 6:00 PM and 8:00 PM.

4.2.6 Results and Interpretation

The resulting choropleth allows the user to select one of the top five qualifying categories and observe its weighted global install distribution.

Countries assigned larger weights receive proportionally larger install estimates and therefore appear with stronger intensity on the choropleth.

For example, the United States has a weight of 0.17 in the manually defined country table, while India has 0.13 and China has 0.11 before normalization. Consequently, these countries receive a larger share of each category's installs than countries assigned smaller weights.

The differences seen geographically therefore result primarily from the **country weighting system**, while differences between categories result from their total Play Store installs.

An important limitation is that the original dataset does not contain actual country-level installation records. Therefore, this visualization should be interpreted as a simulated or weighted global distribution rather than measured geographical Play Store usage.

4.3 Task 3 – Comparison of Free and Paid Applications Based on Installs and Revenue

4.3.1 Objective

The objective of Task 3 was to compare **free and paid applications** across the top three application categories using average installs and average revenue.

The task required the following filters:

- Installs must be **at least 10,000**.
- Paid applications must generate **at least $10,000 revenue**.
- Content rating must be **Everyone**.
- Application name must contain **30 characters or fewer**.

- Application size must be **greater than 15 MB**.
- Required Android version must be **greater than Android 4.0**.
- Application type must be either **Free or Paid**.
- The top three categories should be identified according to total installs.
- The graph should only be available between **1:00 PM and 2:00 PM IST**.

4.3.2 Data Handling

Missing values were removed from the columns needed for the analysis.

The Installs column was cleaned and converted to integer format.

The dollar sign was removed from Price:

```
apps_df['Price'] = (
    apps_df['Price']
    .astype(str)
    .str.replace('$', '', regex=False)
)


apps_df['Price'] = apps_df['Price'].astype(float)
```

Revenue was then calculated as:

```
apps_df['Revenue'] = (
    apps_df['Installs'] *
    apps_df['Price']
)
```

For free applications, price is zero, so calculated revenue is also zero.

Application size was standardized using parse_size_to_mb().

A separate function, is_android_gt_4(), extracted the numerical components of the Android version and determined whether the required version was greater than 4.0.

4.3.3 Filtering According to the Task

Individual Boolean conditions were created:

```
cond_installs = apps_df['Installs'] >= 10000

cond_revenue = (
    (apps_df['Type'] == 'Free') |
    (apps_df['Revenue'] >= 10000)
)

cond_content_rating = (
    apps_df['Content Rating'] == 'Everyone'
)

cond_name_len = (
    apps_df['App'].astype(str).str.len() <= 30
)

cond_size = (
    apps_df['Size'].apply(parse_size_to_mb) > 15
)

cond_android = (
    apps_df['Android Ver'].apply(is_android_gt_4)
)

cond_type = apps_df['Type'].isin(
    ['Free', 'Paid']
```

)

Free applications were intentionally allowed through the revenue condition even though their calculated revenue is zero, because otherwise a free-versus-paid comparison would not be possible.

The conditions were combined using &.

The top three categories were subsequently identified:

```
top3_categories = (
    filtered_df.groupby('Category')['Installs']
    .sum()
    .nlargest(3)
    .index.tolist()
)
```

Average installs and revenue were then calculated for each combination of category and application type.

4.3.4 HTML Graph Code

Plotly was used to create a combined dual-axis visualization.

Bars represented average installs for free and paid applications.

```
go.Bar(
    x=free_data['Category'],
    y=free_data['Installs'],
    name='Free - Avg Installs'
)
```

Revenue was represented using scatter markers:

```
go.Scatter(
    x=paid_data['Category'],
    y=paid_data['Revenue'],
    name='Paid - Avg Revenue',
    mode='markers',
```

```
    yaxis='y2'
)
```

Average installs were displayed on a logarithmic Y-axis because install values can differ substantially in magnitude.

The revenue axis was placed on the right side.

The completed interactive Plotly visualization was exported using:

```
fig.write_html(graph_path)
```

4.3.5 Dashboard Code

The dashboard reported:

- All filtering requirements.
- Total number of filtered applications.
- Top three categories.
- Application type.
- Average installs.
- Average revenue.
- Graph availability.

The graph status depended on whether execution occurred between 1:00 PM and 2:00 PM IST.

4.3.6 Results and Interpretation

The visualization demonstrates the difference in reach and revenue characteristics between free and paid applications.

Free applications can achieve large install counts because users do not need to make an initial payment to download them. Paid applications face an additional purchasing barrier and may therefore have lower average installs.

However, the revenue calculation used in this analysis is:

Revenue = Installs × Price

Consequently, free applications have a calculated revenue of zero because their price is zero.

Paid applications can produce substantial calculated revenue even with fewer installs because every installation is multiplied by the application's purchase price.

The paid-app results are also influenced by the filtering condition requiring paid applications to generate at least $10,000. Low-revenue paid applications are excluded before the category comparison is performed.

Therefore, the visualization represents the characteristics of **qualifying free and paid applications**, not all free and paid applications in the Play Store dataset.

4.4 Task 4 – Time-Series Analysis of Monthly Installs and Month-over-Month Growth

4.4.1 Objective

The objective of Task 4 was to visualize installs over time for selected application categories and identify periods where installs increased by more than **20% compared with the previous available observation**.

The task required:

- Categories beginning with **B, C, or E**.
- The **Dating** category to be included separately.
- Reviews must be **greater than 500**.
- App names must not begin with **X, Y, or Z**.
- App names must not contain uppercase **S**.
- Beauty must be displayed in **Hindi**.
- Business must be displayed in **Tamil**.
- Dating must be displayed in **German**.
- Periods exceeding **20% month-over-month growth** must be highlighted.
- A multi-panel time-series visualization should be generated.
- The visualization should only be available between **6:00 PM and 9:00 PM IST**.

4.4.2 Data Handling

Duplicate rows were removed first.

Missing values in App, Category, Reviews, Installs, and Last Updated were removed.

Installs was converted to integer format after removing commas and plus signs.

Reviews was converted into numerical format.

Last Updated was converted into datetime format.

A list of qualifying categories was created by examining every unique category and retaining categories beginning with B, C, or E together with Dating.

A translation dictionary was created:

```
category_translation_map = {
   'BEAUTY': 'सुंदरता',
   'BUSINESS': 'வணிகம்',
   'DATING': 'Partnersuche'
}
```

4.4.3 Filtering According to the Task

Application-name filtering was handled through a custom function:

```
def is_valid_app_name(name):
   name_str = str(name).strip()

   if name_str.upper().startswith(
      ('X', 'Y', 'Z')
   ):
      return False

   if 'S' in name_str:
      return False

   return True
```

This means the X/Y/Z prefix restriction is case-insensitive, while the S restriction is specifically uppercase and therefore allows lowercase s.

The final conditions were:

```
cond_reviews = apps_df['Reviews'] > 500
cond_category = apps_df['Category'].isin(task4_categories)
cond_app_name = apps_df['App'].apply(is_valid_app_name)
```

The date was subsequently converted into year-month format:

```
task4_filtered_df['Year_Month'] = (
    task4_filtered_df['Last Updated']
    .dt.to_period('M')
)
```

Monthly installs were calculated by category.

Month-over-month growth was calculated using:

```
monthly_installs['MoM_Growth'] = (
    monthly_installs
    .groupby('Category_Display')['Installs']
    .pct_change()
)
```

Values greater than 0.20 were flagged.

4.4.4 HTML Graph Code

A 3 × 3 Plotly subplot arrangement was created:

```
fig = make_subplots(
    rows=3,
    cols=3,
    subplot_titles=subplot_titles
)
```

Each category received its own time-series panel.

Normal install values were represented using a line:

```
go.Scatter(
    x=cat_df['Year_Month_DT'],
    y=cat_df['Installs'],
    mode='lines'
)
```

When month-over-month growth exceeded 20%, a shaded polygon was added between the previous and current observations.

```
if curr_row['MoM_Growth'] > 0.20:
    fig.add_trace(
        go.Scatter(
            x=[...],
            y=[...],
            fill='toself',
            fillcolor='rgba(255, 165, 0, 0.25)'
        )
    )
```

If more than nine categories qualified, additional 3 × 3 pages were generated.

Each Plotly figure was converted to an HTML fragment using:

```
fig.to_html(
    full_html=False,
    include_plotlyjs='cdn'
)
```

The fragments were combined into task_4_graph.html.

4.4.5 Dashboard Code

The dashboard summarized:

- Review threshold.
- Category restrictions.
- Application-name restrictions.
- Number of qualifying rows.
- Localized category names.
- Combined installs by category.
- Graph status.

The graph was only generated between 6:00 PM and 9:00 PM IST.

4.4.6 Results and Interpretation

The resulting visualization shows how install totals associated with different Last Updated months vary among the selected categories.

Periods shaded in orange represent increases exceeding 20% compared with the previous available observation for the same category.

A sharp increase can occur when applications associated with a particular month collectively have substantially more installs than applications associated with the preceding available observation.

An important point is that pct_change() is performed on the available rows for each category. Therefore, if a category has observations for January and March but none for February, the March value is compared with January rather than with a zero-valued February observation.

The graph should also not be interpreted as a true record of installations that occurred during each historical month. The original Play Store dataset provides an application's Last Updated date and current install count rather than monthly historical install events.

Therefore, this analysis groups current install totals according to the month in which applications were last updated. The visualization is useful for comparing update-period patterns but does not represent a direct historical download log.

4.5 Task 5 – Bubble Chart Analysis of App Size, Rating, Installs and Sentiment Subjectivity

4.5.1 Objective

The objective of Task 5 was to create an interactive **bubble chart** examining the relationship between application size and rating while representing the number of installs through bubble size.

The required conditions were:

- Rating must be **greater than 3.5**.
- Reviews must be **greater than 500**.
- Installs must be **greater than 50,000**.
- Average sentiment subjectivity must be **greater than 0.5**.
- Applications must belong to:
    - Game
    - Beauty

    - Business
    - Comics
    - Communication
    - Dating
    - Entertainment
    - Social
    - Events
- Application names must not contain uppercase **S**.
- Game must be highlighted in **pink**.
- Beauty must be translated into **Hindi**.
- Business must be translated into **Tamil**.
- Dating must be translated into **German**.
- The graph should only appear between **5:00 PM and 7:00 PM IST**.

4.5.2 Data Handling

Both Play Store Data.csv and User Reviews.csv were required.

The Play Store dataset was cleaned by removing missing values and converting installs, reviews, ratings, and application sizes into usable numerical formats.

Ratings greater than 5 were also removed because they are invalid for the Google Play rating scale.

The Sentiment_Subjectivity field from the User Reviews dataset was converted to numeric form.

Average subjectivity was calculated separately for every application:

```
avg_subjectivity_df = (
  reviews_df.groupby('App')[
    'Sentiment_Subjectivity'
  ]
  .mean()
  .reset_index()
```

```
)
```

It was renamed to Avg_Subjectivity.

The result was then merged with the Play Store dataset:

```
merged_df = pd.merge(
    apps_df,
    avg_subjectivity_df,
    on='App',
    how='inner'
)
```

An inner join was appropriate because the final analysis required both Play Store information and sentiment subjectivity information.

4.5.3 Filtering According to the Task

The target categories were explicitly defined:

```
target_categories = [
    'GAME', 'BEAUTY', 'BUSINESS',
    'COMICS', 'COMMUNICATION', 'DATING',
    'ENTERTAINMENT', 'SOCIAL', 'EVENTS'
]
```

The conditions were:

```
cond_rating = merged_df['Rating'] > 3.5


cond_category = (
    merged_df['Category']
    .str.upper()
    .isin(target_categories)
)
```

```
cond_reviews = merged_df['Reviews'] > 500

cond_installs = merged_df['Installs'] > 50000


cond_subjectivity = (
    merged_df['Avg_Subjectivity'] > 0.5
)


cond_no_s = ~merged_df['App'].astype(str).str.contains(
    'S',
    case=True,
    na=False
)
```

Because case=True is used, only uppercase S is prohibited. An application name containing lowercase s can remain.

The translation dictionary was:

```
category_translation_map = {
    'BEAUTY': 'सुंदरता',
    'BUSINESS': 'வணிகம்',
    'DATING': 'Partnersuche'
}
```

4.5.4 HTML Graph Code

Plotly was used to create the bubble chart.

The X-axis represented application size:

```
x=category_df['Size_MB']
```

The Y-axis represented rating:

```
y=category_df['Rating']
```

Bubble size represented installs:

```
marker=dict(
    size=category_df['Installs'],
    sizemode='area',
    sizeref=sizeref,
    sizemin=6
)
```

The Game category was highlighted using:

```
marker_color = (
    'pink'
    if category == 'GAME'
    else None
)
```

Interactive hover information included:

- Application name
- Category
- Size
- Rating
- Installs
- Reviews
- Average sentiment subjectivity

The completed interactive graph was saved using:

```
fig.write_html(graph_path)
```

4.5.5 Dashboard Code

The dashboard displayed:

- Filtering criteria.
- Total filtered rows.
- Category name.

- Number of applications.
- Average rating.
- Total installs.
- Graph availability status.

The graph was generated only between 5:00 PM and 7:00 PM IST.

4.5.6 Results and Interpretation

The bubble chart allows three major numerical variables to be interpreted simultaneously.

Horizontal position represents application size.

Vertical position represents rating.

Bubble size represents installs.

Therefore, a large bubble indicates a highly installed application, while its position reveals its size and rating.

Because applications with ratings of 3.5 or below have already been removed, all points appear in the upper portion of the Google Play rating scale.

Likewise, small or unpopular applications are reduced by the reviews >500 and installs >50,000 requirements.

The subjectivity filter produces an additional important effect. Only applications with an average sentiment subjectivity greater than 0.5 are retained. Subjectivity describes how opinion-based rather than purely objective the review text is. Therefore, the graph specifically represents applications whose available reviews have relatively high average subjectivity.

The visualization can also be used to determine whether larger applications necessarily receive higher ratings. Bubble positions may demonstrate that application size alone does not determine user satisfaction. Similarly, very large bubbles can appear at different ratings because popularity and rating measure different aspects of application performance.

The pink Game bubbles provide an immediate visual reference for comparing Game applications with the remaining selected categories.

4.6 Task 6 – Cumulative Installs by Category Using a Stacked Area Chart

4.6.1 Objective

The objective of Task 6 was to create a **stacked area chart showing cumulative installs over time for each qualifying application category**.

The required filters were:

- Rating must be **at least 4.2**.
- Application name must **not contain numerical digits**.
- Category must begin with **T or P**.
- Reviews must be **greater than 1,000**.
- Application size must be between **20 MB and 80 MB**.
- Travel & Local must be translated into **French**.
- Productivity must be translated into **Spanish**.
- Photography must be translated into **Japanese**.
- Months where installs increased by more than **25%** compared with the previous available category observation should be displayed with stronger color intensity.
- The graph should only be displayed between **4:00 PM and 6:00 PM IST**.

4.6.2 Data Handling

Duplicate records were removed.

Rows containing missing values in required columns were removed.

The Installs column was cleaned by removing commas and plus signs and converting the result into integers.

Rating and Reviews were converted into numeric form.

Application size was converted into megabytes using parse_size_to_mb().

Last Updated was converted into datetime format.

After conversion, rows where rating, reviews, size, or date could not be successfully interpreted were removed.

The month of the last update was extracted:

filtered_df['Month'] = (

```
    filtered_df['Last Updated']
    .dt.to_period('M')
    .dt.to_timestamp()
)
```

4.6.3 Filtering According to the Task

Five individual filtering conditions were created:

```
cond_rating = apps_df['Rating'] >= 4.2
cond_no_numbers = ~apps_df['App'].astype(str).str.contains(
    r'\d',
    regex=True,
    na=False
)
```

\d represents any numerical digit. The ~ operator reverses the condition, meaning applications containing digits are excluded.

Categories beginning with T or P were selected using:

```
cond_category = (
    apps_df['Category']
    .astype(str)
    .str.startswith(('T', 'P'))
)
```

The remaining conditions were:

```
cond_reviews = apps_df['Reviews'] > 1000


cond_size = apps_df['Size_MB'].between(
    20,
    80
)
```

The conditions were combined using Boolean AND operations.

Monthly installs were then calculated:

```
monthly_df = (
    filtered_df
    .groupby(['Month', 'Category'])['Installs']
    .sum()
    .reset_index()
)
```

Growth was calculated separately for each category:

```
monthly_df['Growth'] = (
    monthly_df
    .groupby('Category')['Installs']
    .pct_change() * 100
)
```

Growth exceeding 25% was flagged:

```
monthly_df['High_Growth'] = (
    monthly_df['Growth'] > 25
)
```

The data was reshaped into a Month × Category table:

```
pivot_df = monthly_df.pivot(
    index='Month',
    columns='Category',
    values='Installs'
).fillna(0).sort_index()
```

fillna(0) was used here because missing category-month combinations need a numerical contribution for cumulative plotting. It does **not** change the earlier growth calculation.

Cumulative installs were then calculated:

```
cumulative_df = pivot_df.cumsum()
```

4.6.4 HTML Graph Code

Matplotlib's stackplot() was used:

```
ax.stackplot(
    dates,
    [
        cumulative_df[category].values
        for category in category_order
    ],
    labels=labels,
    colors=[
        category_colors[category]
        for category in category_order
    ],
    alpha=0.45
)
```

Each category therefore appears as a separate colored band.

Normal areas use an opacity of:

```
alpha=0.45
```

Qualifying high-growth regions are redrawn with:

```
alpha=0.95
```

This produces the stronger color intensity requested by the task.

The vertical boundaries of every stacked band were calculated using:

```
stack_top = cumulative_df.cumsum(axis=1)
stack_bottom = stack_top - cumulative_df
```

fill_between() was then used to overlay the stronger category color where the high-growth condition was satisfied.

The completed Matplotlib figure was saved into memory, converted to Base64, and embedded in task_6_graph.html.

The generated graph itself states that darker areas indicate months with more than 25% month-over-month growth.

4.6.5 Dashboard Code

The Task 6 dashboard contains a summary of the filtering criteria and reports the total number of filtered rows.

The generated dashboard shows that **129 rows remained after filtering**.

The category summary table contains:

- Category
- Number of applications
- Average rating
- Total installs

The final dashboard produced the following values:

Category	Apps	Average Rating	Total Installs
Parenting	4	4.62	10,700,000
Personalization	11	4.49	64,500,000
Photography (写真)	49	4.42	1,594,500,000
Productivity (Productividad)	20	4.40	1,987,100,000
Tools	17	4.49	353,650,000
Travel & Local (Voyages et infos locales)	28	4.38	94,250,000

These values are taken directly from the generated Task 6 dashboard.

The dashboard also contains the graph-status section. In the supplied final output, the graph was successfully generated because execution occurred within the permitted 4:00 PM–6:00 PM IST period.

4.6.6 Results and Interpretation

Task 6 provides one of the clearest examples of how filtering and aggregation directly affect a visualization.

After all conditions were applied, 129 qualifying records remained. The largest install total belonged to **Productivity**, with approximately **1.987 billion installs**, followed by **Photography**, with approximately **1.595 billion installs**.

This explains why Productivity and Photography make much larger contributions to the stacked area chart than categories such as Parenting and Personalization.

Photography also has the greatest number of qualifying applications, with 49 applications, whereas Productivity contains 20. However, Productivity still has a greater total install count.

This demonstrates an important analytical point:

A category does not need to contain the largest number of applications to have the largest number of installs.

The average qualifying Productivity application contributes considerably more installs than would be suggested merely by comparing application counts.

Parenting has only four qualifying applications and 10.7 million total installs. Consequently, its contribution to the cumulative stacked area is comparatively small.

The average ratings, on the other hand, are relatively similar. They range from approximately 4.38 to 4.62 in the final dashboard. This narrow range is expected because the filtering stage already removed every application with a rating below 4.2.

Therefore, large differences in the graph are mainly caused by differences in install totals rather than rating.

Why Does the Graph Continuously Increase?

The chart uses:

cumulative_df = pivot_df.cumsum()

Each month's value is added to everything accumulated previously.

For example, if a category contributes:

- January = 100 million
- February = 50 million
- March = 75 million

the cumulative values become:

- January = 100 million
- February = 150 million
- March = 225 million

Therefore, the cumulative line cannot decrease simply because a later monthly contribution is smaller. A zero contribution leaves the cumulative total unchanged rather than reducing it.

Why Are Some Areas Darker?

The normal stacked areas use lower opacity, while months identified as having growth greater than 25% are overlaid with higher opacity.

The darker regions therefore identify periods where the monthly install total associated with a category increased by more than 25% compared with its previous available category observation.

The darker area does **not** mean that the category has the greatest total installs overall. It specifically represents the growth condition.

Why Are There Missing Months for Some Categories?

Not every qualifying category necessarily has applications associated with every Last Updated month.

After pivoting, such combinations appear as NaN.

These values are changed to zero using:

.fillna(0)

This means that the category contributes zero additional installs for that particular chart month. Because the cumulative total is calculated afterward, the previous cumulative amount is carried forward.

Interpretation Limitation

As with Task 4, the time dimension requires careful interpretation.

The Google Play Store dataset does not provide a historical record stating exactly how many installations occurred during each month.

Instead, the notebook groups applications according to their **Last Updated month** and sums their current install values.

Therefore, the stacked area chart should be described as cumulative installs associated with applications grouped by their last-update periods rather than a literal historical record of downloads occurring month by month.

5. Overall Learning From the Six Tasks

The six tasks collectively provided experience with increasingly complex data-analysis workflows.

Task 1 introduced numerical cleaning, aggregation, ranking, grouped bars, and secondary axes.

Task 2 introduced interactive geographical visualization and weighted data distribution.

Task 3 introduced price cleaning, calculated revenue, free-versus-paid comparison, custom Android-version filtering, and logarithmic visualization.

Task 4 introduced time-series aggregation, localized labels, month-over-month growth calculations, conditional highlighting, and multi-panel visualizations.

Task 5 introduced merging two datasets, sentiment subjectivity, multidimensional bubble charts, interactive hover information, and conditional category highlighting.

Task 6 integrated filtering, datetime processing, pivot tables, missing category-month handling, cumulative calculations, stacked area visualization, growth highlighting, translations, HTML generation, and dashboard reporting.

Together, the tasks demonstrated a complete analytical workflow:

Data Loading → Data Cleaning → Transformation → Filtering → Aggregation → Analysis → Visualization → HTML Generation → Dashboard Presentation → Interpretation

The most important learning outcome was that the final visualization is determined not only by the plotting library but also by every decision made during preprocessing and filtering. Understanding why a graph has a particular shape therefore requires understanding how the underlying data was selected, transformed, grouped, and calculated.

6. Skills and Competencies

The internship contributed to the development of both technical and analytical competencies.

Python Programming

Regular use of Python improved familiarity with variables, functions, loops, conditional statements, dictionaries, list comprehensions, exception handling, and logical operators.

Data Cleaning

Considerable experience was gained in preparing raw data before analysis. Important operations included removing duplicate records, handling missing values, converting data types, removing unwanted characters, and standardizing application-size values.

For example, install values containing commas and plus signs were converted into integers before mathematical operations were performed.

Pandas

Pandas was one of the primary tools used throughout the internship. Important methods and concepts included:

dropna(), drop_duplicates(), groupby(), agg(), merge(), pivot(), sort_values(), nlargest(), isin(), between(), pct_change(), cumsum(), and datetime operations.

Data Visualization

Both Matplotlib and Plotly were used to develop visualizations. Experience was gained in creating grouped bar charts, choropleth maps, combined bar-and-marker visualizations, time-series graphs, bubble charts, stacked area charts, and multi-panel plots.

Data Integration

The sentiment-analysis task provided practical experience in combining two datasets. Average sentiment subjectivity was calculated from the user review data and merged with application information.

Time-Series Analysis

The later tasks improved understanding of datetime conversion, monthly aggregation, chronological sorting, month-over-month growth, and cumulative calculations.

Dashboard Development

HTML and CSS were used together with Python-generated content to create simple dashboards. The dashboards displayed filtering information, category summaries, analytical results, and links to generated graphs.

Timezone-Based Programming

The pytz library and Python datetime functionality were used to determine the current time in the Asia/Kolkata timezone. Conditional logic was then used to control whether a visualization should be generated and displayed during the permitted time window.

Problem Solving

The tasks required translating written analytical requirements into programming logic. This improved the ability to break a complex problem into smaller steps such as cleaning, filtering, grouping, calculating, visualizing, and validating the final result.

7. Feedback and Evidence

Evidence of learning during the internship was demonstrated through the successful completion of six analytical notebooks and the corresponding visualization and dashboard outputs.

Each task provided evidence of progressive development. The earlier tasks concentrated on cleaning, filtering, grouping, and relatively straightforward category comparisons. Later tasks required more complex techniques such as dataset merging, multilingual labels, sentiment subjectivity, time-series analysis, month-over-month growth, cumulative calculations, and conditional highlighting.

The generated dashboards also served as evidence that the analytical results could be converted into presentable outputs rather than remaining only as Python code.

The final task, for example, successfully produced a dashboard containing 129 filtered records and displayed category-level summaries. The output included the required translated category names, including Japanese for Photography, Spanish for Productivity, and French for Travel & Local.

Feedback received during the development process was incorporated through repeated review and correction of the code. This process improved the accuracy of filtering logic, graph formatting, axis representation, category selection, time restrictions, and overall presentation.

The iterative nature of the work demonstrated that data analysis is not only about producing an initial result but also about validating whether the result accurately satisfies the given requirements.

8. Challenges and Solutions

Several challenges were encountered during the internship.

Handling Inconsistent Data Formats

Columns such as Installs and Size could not always be used directly for numerical calculations.

Solution: String-processing techniques were used to remove commas and plus signs from install values. A custom function was created to convert sizes represented in megabytes and kilobytes into a common MB format.

Managing Missing Values

Missing values could interfere with filtering, aggregation, and visualization.

Solution: Relevant rows were removed or converted using methods such as dropna() and pd.to_numeric(..., errors='coerce').

Applying Multiple Filtering Conditions

Many tasks required several conditions to be satisfied simultaneously.

Solution: Individual Boolean conditions were created and combined using logical operators such as &, |, and ~. Separating the conditions also made the code easier to understand and debug.

Working With Dates

The Last Updated column originally contained date information that needed to be converted before monthly analysis.

Solution: pd.to_datetime() was used to create datetime values, followed by period conversion to represent year and month.

Calculating Month-over-Month Growth

Growth had to be calculated separately for each application category rather than across the entire dataset.

Solution: Data was first sorted according to category and month, and groupby() with pct_change() was used to compare each category with its previous available observation.

Combining Two Datasets

The bubble-chart task required sentiment information from a separate User Reviews dataset.

Solution: Average sentiment subjectivity was calculated for each app and merged with the Play Store data using the common App field.

Visualizing Variables With Different Scales

Variables such as ratings, reviews, installs, and revenue differed greatly in magnitude.

Solution: Secondary Y-axes, logarithmic scaling, and bubble-size encoding were used where appropriate to present variables more clearly.

Highlighting Growth Conditions

Certain tasks required periods exceeding a specified month-over-month growth threshold to be visually distinguishable.

Solution: Boolean growth flags were created, and additional plotting elements with stronger opacity or shading were placed over qualifying periods.

Implementing Time Restrictions

Graphs were required to appear only during particular IST time windows.

Solution: Timezone-aware current time was obtained using pytz.timezone('Asia/Kolkata'). Boolean conditions were then used to determine whether graph generation and dashboard display should occur.

9. Outcomes and Impact

The internship resulted in the successful completion of six Python-based data analysis and visualization tasks.

A major outcome was the development of an end-to-end understanding of the data analysis process. The internship demonstrated that creating a meaningful visualization requires much more than selecting a graph type. Data must first be understood, cleaned, converted into appropriate formats, filtered correctly, aggregated according to the analytical objective, and validated before visualization.

The internship also improved confidence in using Pandas for data manipulation and Matplotlib and Plotly for visualization.

Working with different graph types helped develop an understanding of when different visualization approaches are appropriate. Bar charts were useful for category comparisons, choropleth maps for geographical representation, line charts for changes over time, bubble charts for representing multiple variables simultaneously, and stacked area charts for cumulative category-level trends.

The dashboard-development component further demonstrated how analytical results can be communicated in a structured and user-friendly format.

The project also improved debugging and problem-solving abilities. Complex requirements had to be translated into precise Python conditions, and errors or unexpected results required examination of intermediate DataFrames rather than only the final visualization.

Overall, the internship strengthened practical competencies that can be applied to future data analytics, data science, bioinformatics, and research projects involving large datasets.

10. Conclusion

The internship provided valuable practical experience in Python programming, data preprocessing, exploratory data analysis, visualization, and dashboard development.

Through six progressively complex tasks based on Google Play Store data, multiple stages of a complete data-analysis workflow were implemented. These included loading datasets, cleaning raw data, handling missing values, transforming columns, filtering records, aggregating information, merging datasets, calculating statistical measures, analyzing time-dependent changes, and generating graphical outputs.

The internship also provided hands-on experience with important Python libraries such as Pandas, NumPy, Matplotlib, Plotly, datetime, and pytz. Advanced concepts such as month-over-month growth, cumulative calculations, pivot tables, sentiment subjectivity, multilingual labels, geographical visualization, secondary axes, interactive plots, and timezone-based conditions were also explored.

One of the most important lessons gained from the internship was the importance of data preparation. The quality and accuracy of a visualization depend directly on how correctly the underlying data has been cleaned, filtered, transformed, and aggregated.

The internship therefore contributed not only to technical programming skills but also to analytical thinking, logical problem solving, debugging, and the ability to communicate data through meaningful visualizations.

In conclusion, the internship successfully provided practical exposure to real-world data analysis and strengthened the foundation required for future work in data analytics, data science, bioinformatics, and other computational research fields.